

TAN TAO UNIVERSITY
SCHOOL OF INFORMATION TECHNOLOGY



TAN TAO UNIVERSITY
FROM KNOWLEDGE TO THE STARS

Report of Team Project 2
SOCKET PROGRAMMING WITH PYTHON
A MULTI-USER CHAT APPLICATION

Course: Computer Networks (CS440V)

Spring 2026

GROUP MEMBER

Member 1: Ho Nguyen Kim Long	Student code: 2102161
Member 2: Le Chi Tam	Student code: 2302220
Member 3: Trieu Minh Thuan	Student code: 2302059
Member 4: Le Nguyen Quoc Anh	Student code: 2302283

Supervisor: Truong Huu Tram, PhD and Le Quoc Huy, PhD

Github: https://github.com/MrNhoiBong/Chatting_Encryption

Contents

1	Introduction	1
2	Project Overview	2
2.1	Application Description	2
2.2	Main Features	3
2.3	Technologies Used	3
2.4	Overall Architecture	4
3	System Design	5
3.1	Architecture Overview	5
3.2	Encryption Design	6
3.2.1	Identity-Based Encryption (IBE) – Boneh-Franklin Scheme	6
3.2.2	Hybrid Encryption per Message	6
3.2.3	Group Chat Encryption	7
3.3	Communication Workflow	7
3.3.1	User Registration Flow	7
3.3.2	Private Messaging Flow	8
3.3.3	Group Chat Flow	8
3.4	Concurrency Handling	8
3.5	Data Transmission Format	9
3.6	Deployment Architecture	9
4	Implementation	10
4.1	File Structure and Purpose	10
4.1.1	ServerContainer	10
4.1.2	ClientContainer	10
4.2	Server Implementation (server_api.py)	11
4.2.1	Initialization	11
4.2.2	Message Dispatch in handle_client	11
4.3	Client Backend Implementation (client_api.py)	12
4.3.1	Global State	12

4.3.2	Background Listener Thread	13
4.3.3	REST API Endpoints	14
4.4	Frontend Implementation (App.jsx)	15
4.4.1	State Management	15
4.4.2	Polling Loop	15
4.4.3	Group Chat UI Components	15
5	Challenges	17
5.1	Socket Connection and Message Framing	17
5.2	Multi-Threading Challenges	17
5.3	Identity-Based Encryption (IBE) Serialization	18
5.4	Group Chat Implementation Challenges	18
6	Future Improvements	20
6.1	Attribute-Based Encryption (ABE) for Fine-Grained Access Control	20
6.2	Persistent Message Storage	20
6.3	Multi-Server Architecture and Load Balancing	21
6.4	Decentralized Key Management with Multi-Authority ABE	21
6.5	WebSocket-Based Real-Time Frontend	21
6.6	Group Management Features	21
7	Conclusion	23

Chapter 1

Introduction

The objective of this project is to design and implement a socket-based application that enables secure, real-time communication in a client-server environment. Unlike traditional chat systems, this project integrates **Identity-Based Encryption (IBE)** using the Boneh-Franklin scheme to ensure that all exchanged messages are cryptographically protected and that each participant is authenticated based on their unique identity string, without the need for a traditional Public Key Infrastructure (PKI).

The system supports two modes of communication: **one-to-one (private) messaging** and **group chat**. In group chat mode, the server acts as a Private Key Generator (PKG) and issues a dedicated secret key for each group identity. Group members encrypt messages using the group's identity as the IBE public key, and any member who holds the corresponding group secret key can decrypt the message. This design allows secure multi-party communication without sharing symmetric keys out-of-band.

The report focuses on the practical realization of this system, highlighting the architectural decisions, implementation strategies, and technical considerations that make such a system feasible. The scope does not extend to broader theoretical discussions of cryptography but instead emphasizes how socket programming, multi-threading, containerized deployment, and IBE work together to form a cohesive and secure chat application.

This introduction sets the stage for the subsequent chapters: the project overview, system design, implementation details (including the new group chat functionality), encountered challenges, potential future improvements, and final conclusions.

Chapter 2

Project Overview

The project focuses on building a secure, multi-user chat application using socket programming in Python. The system supports both **one-to-one private messaging** and **group chat**, all protected end-to-end using Identity-Based Encryption (IBE) combined with AES symmetric encryption. The architecture separates the server, client backend, and client frontend into distinct, containerized components, enabling modular development and deployment.

2.1 Application Description

- **Client-server model:** The system follows a client-server architecture where the server acts both as a message relay and as a Private Key Generator (PKG) for the IBE scheme.
- **One-to-one private messaging:** Users can send encrypted messages directly to another online user. The message is encrypted using the recipient's identity string as the IBE public key, and only the recipient's secret key can decrypt it.
- **Group chat:** Users can create named chat groups, invite other users, and communicate securely within the group. The server generates a dedicated IBE secret key for each group identity; all group members share this key and can decrypt group messages.
- **Graphical user interface (GUI):** A browser-based React frontend provides a modern, user-friendly interface for connecting, chatting, creating groups, and managing invitations.
- **Client backend:** A Flask-based Python service runs in the background, maintaining a persistent TCP socket connection with the server, performing all cryptographic operations, and exposing REST endpoints for the frontend.

2.2 Main Features

- **Identity-Based Encryption (IBE):** Ensures confidentiality and authentication by binding encryption keys to user or group identity strings. No certificate authority is required.
- **Hybrid encryption (IBE + AES):** A fresh 256-bit AES session key is generated per message. The session key is encrypted with IBE; the message content is encrypted with AES-EAX, providing both confidentiality and integrity.
- **Real-time private messaging:** Encrypted payloads are forwarded immediately to the intended recipient's socket without server-side storage.
- **Group creation and invitation:** Any connected user can create a group, invite online users, and communicate with all group members simultaneously.
- **Group message broadcasting:** The server routes encrypted group messages to all members of the group who are currently online.
- **Invitation management:** Invited users receive a real-time pop-up notification and can accept or decline group invitations.
- **Online user discovery:** The server broadcasts the current user list to all connected clients whenever a user connects or disconnects.
- **Connection persistence check:** The client backend exposes a `/status` endpoint so the frontend can resume a session without re-entering credentials.

2.3 Technologies Used

- **Programming language:** Python 3 for both the server and client backend.
- **Frontend framework:** React (with Vite) and Tailwind CSS for the browser-based user interface.
- **Networking:** Python socket module for raw TCP communication; threading for concurrent connection handling.
- **API layer:** Flask with Flask-CORS exposes RESTful HTTP endpoints for the frontend.
- **Cryptography:** Charm-Crypto library (Boneh-Franklin IBE over the SS512 pairing group) and PyCryptodome (AES-EAX).
- **Containerization:** Docker provides isolated Linux environments required by the charm-crypto pairing library.

- **Data serialization:** JSON is used for all message payloads; binary cryptographic data is Base64-encoded before JSON serialization.

2.4 Overall Architecture

- **Server (API Server):** A multi-threaded TCP socket server. It registers users, acts as the PKG by generating and distributing IBE secret keys, maintains the online user list, routes private messages, manages group membership, and broadcasts group messages to all online members.
- **Client Backend (API Client):** A Flask application that connects to the server via TCP, registers the user, receives its IBE secret key, and starts a background listener thread. It encrypts outgoing messages locally and decrypts incoming messages before storing them in an in-memory buffer accessible to the frontend.
- **Client Frontend:** A React single-page application served via Vite. It polls the client backend every two seconds for new messages, online users, group information, and pending invitations, and renders the results in a modern chat interface.

Chapter 3

System Design

The system is designed following a three-tier client-server architecture, where the server manages multiple socket connections concurrently, routes encrypted messages between individual users and groups, and fulfills the role of a Private Key Generator (PKG) in the Identity-Based Encryption scheme. The design emphasizes modularity by separating responsibilities into distinct components, each deployed in its own Docker container.

3.1 Architecture Overview

The system consists of three main tiers:

- **API Server (ServerContainer):** A raw TCP socket server written in Python. It acts as the central hub for all connections and is responsible for:
 - Accepting incoming TCP connections from client backends.
 - Running the IBE setup (Boneh-Franklin over SS512) at startup to generate the Master Public Key (MPK) and Master Secret Key (MSK).
 - Acting as a PKG: when a user registers, the server extracts and delivers an IBE secret key bound to the user's identity. When a group is created, the server extracts and delivers a secret key bound to the group name as the group's identity.
 - Maintaining the active client registry (`clients`) and group registry (`groups`) in memory, protected by a shared threading lock.
 - Routing private messages directly to the recipient's socket.
 - Broadcasting group messages to all online members of the target group.
 - Broadcasting the updated user list to all connected clients on any join or leave event.
- **API Client (ClientContainer/API_client):** A Flask application exposing a REST API, which:

- Opens and maintains a persistent TCP connection to the API Server.
 - Registers the user and receives the MPK and its personal IBE secret key (SK).
 - Runs a background listener thread (`listen_for_messages`) that continuously reads push-delivered data from the server socket and dispatches it based on message type.
 - Performs all local cryptographic operations: encrypting outgoing messages (IBE + AES-EAX) and decrypting incoming messages using the appropriate secret key.
 - Manages in-memory state: the received message buffer, the active user list, joined groups with their respective group secret keys, and pending group invitations.
 - Exposes REST endpoints that the React frontend calls via HTTP.
- **Frontend (ClientContainer/frontend):** A React single-page application that:
 - Provides a login screen where the user enters their identity name, the server IP address, and the server port.
 - Polls the client backend every 2 seconds for new messages, online users, group membership changes, and pending invitations.
 - Renders separate sidebar sections for online users (private chat) and joined groups (group chat).
 - Displays real-time pop-up notifications for incoming group invitations, allowing the user to accept or decline.
 - Displays system error notifications (e.g., offline recipient) as dismissible toast messages.

3.2 Encryption Design

3.2.1 Identity-Based Encryption (IBE) – Boneh-Franklin Scheme

The IBE scheme eliminates the need for certificate exchange. The server's PKG function generates a single master key pair (MPK, MSK) at startup. All clients receive the same MPK. Each user's secret key SK_{id} is derived from the MSK and the user's identity string id using $\text{Extract}(MSK, id) \rightarrow SK_{id}$.

3.2.2 Hybrid Encryption per Message

Each outgoing message (private or group) uses the following hybrid scheme:

1. A fresh 256-bit AES session key K is generated using a cryptographically secure random number generator.

2. The plaintext message is encrypted with AES-EAX mode: $(C, \text{tag}, \text{nonce}) \leftarrow \text{AES-EAX.Encrypt}(K, m)$. EAX provides both confidentiality and integrity (authentication tag).
3. The AES session key K is encrypted with IBE using the recipient's identity id_r as the public key: $C_K \leftarrow \text{IBE.Encrypt}(MPK, id_r, K)$.
4. The final payload sent over TCP is a JSON object containing C_K (Base64-encoded IBE ciphertext components), the AES nonce, ciphertext, and authentication tag.

On the receiver side, the client backend decrypts in reverse: $K \leftarrow \text{IBE.Decrypt}(MPK, SK_{id_r}, C_K)$, then $m \leftarrow \text{AES-EAX.Decrypt}(K, C, \text{nonce}, \text{tag})$.

3.2.3 Group Chat Encryption

For group chat, the group name (e.g., "CS440_Group") is used as the IBE identity for the group:

1. When a group is created, the server computes $SK_{group} \leftarrow \text{Extract}(MSK, \text{group_name})$ and sends it to the creator.
2. When a user accepts a group invitation, the server again derives SK_{group} and sends it to the new member.
3. Any group member who wants to send a message encrypts the AES session key using $\text{IBE.Encrypt}(MPK, \text{group_name}, K)$.
4. Since all group members possess the same SK_{group} , any member can decrypt the AES session key and recover the plaintext.

This design allows the server to broadcast a single encrypted payload to all online group members; each member independently decrypts it using their locally stored SK_{group} .

3.3 Communication Workflow

3.3.1 User Registration Flow

1. The client backend connects via TCP and sends $\{\text{"type": "register", "username": "Alice"}\}$.
2. The server checks for username uniqueness, registers the client, derives SK_{Alice} , and responds with the MPK, SK_{Alice} , and the current online user list.
3. The client backend starts the background listener thread and notifies the frontend of a successful connection.

3.3.2 Private Messaging Flow

1. The frontend calls `POST /send` with the target username and plaintext.
2. The client backend performs hybrid encryption (AES + IBE on id_{target}) and sends the payload via TCP.
3. The server reads the payload, looks up the recipient's socket, and forwards the encrypted JSON without modification.
4. The recipient's listener thread receives the payload, decrypts it using $SK_{recipient}$, and stores the plaintext in the local message buffer.
5. The recipient's frontend retrieves the message via `GET /messages` in the next polling cycle.

3.3.3 Group Chat Flow

1. **Create group:** User sends `POST /groups/create` with `group_name`. The client backend forwards a `create_group` socket message. The server derives SK_{group} and returns it. The client backend stores SK_{group} in `local_group_sks`.
2. **Invite member:** The group creator sends `POST /groups/invite` with the target username. The server delivers a `group_invitation` push message to the target's socket.
3. **Accept invitation:** The invitee's frontend displays a pop-up. Upon acceptance, `POST /groups/accept` is called. The server derives SK_{group} and delivers it. The invitee's backend stores SK_{group} .
4. **Send group message:** The sender calls `POST /groups/send`. The client backend encrypts the AES session key with `IBE.Encrypt(MPK, group_name, K)` and sends a `group_message` payload via TCP. The server iterates over all online group members (excluding the sender) and forwards the encrypted payload to each.
5. **Receive group message:** Each member's listener thread decrypts the session key using SK_{group} , decrypts the message content, and appends it to the local message buffer with the `is_group: true` flag. The frontend filters messages by group name and renders them in the correct chat window.

3.4 Concurrency Handling

- The server spawns a dedicated daemon thread per client connection (`handle_client`). All threads share the `clients` and `groups` dictionaries, protected by a single `threading.Lock()` to prevent race conditions during reads and writes.

- The client backend runs a dedicated daemon listener thread (`listen_for_messages`) alongside the main Flask thread. Shared in-memory structures (`local_messages_db`, `active_users`, `local_group_sks`, `pending_invitations`, `joined_groups`) are accessed under a `threading.Lock()` named `config_lock`.
- Flask is started with `threaded=True`, allowing multiple concurrent frontend HTTP requests to be handled without blocking the socket listener thread.

3.5 Data Transmission Format

All messages exchanged between the server and client backends are newline-delimited JSON objects transmitted as UTF-8 byte streams over TCP. Since TCP does not preserve message boundaries, both sides implement a streaming JSON parser (`json.JSONDecoder().raw_decode`) with a string buffer to correctly reassemble fragmented or concatenated JSON objects. Binary cryptographic values (IBE ciphertext elements, AES nonce, ciphertext, tag) are Base64-encoded before embedding in JSON.

3.6 Deployment Architecture

Each container is isolated using Docker:

- **ServerContainer:** Runs `server_api.py` on TCP port 5000. Accessible from all client containers.
- **ClientContainer:** Runs both the Flask API Client (port 5001) and the Vite frontend (port 5173) together. The frontend communicates with the client backend on `localhost:5001`; the client backend connects to the server using the IP and port entered by the user (typically `host.docker.internal:5000` in Docker Desktop environments).

Chapter 4

Implementation

4.1 File Structure and Purpose

The project is organized into two top-level directories corresponding to the two Docker containers.

4.1.1 ServerContainer

- **API_server/**
 - `server_api.py`: The complete server implementation. Initializes the IBE system, listens for TCP connections, spawns a handler thread per client, dispatches all message types (`register`, `message`, `create_group`, `invite_group`, `join_group`, `group_message`, `request_users`), and manages in-memory state.
 - `Dockerfile`: Builds the server container with `charm-crypto` and its pairing library dependencies.
 - `README_API.md`: Instructions for running the server container.
 - `requirements.txt`: Python dependencies (`charm-crypto`, etc.).
- `Dockerfile`: Main Dockerfile for the server container.
- `README.md`: General documentation for the ServerContainer.

4.1.2 ClientContainer

- **API_client/**
 - `client_api.py`: The client backend. A Flask application that connects to the server, manages local cryptographic state, runs a background socket listener, and exposes all REST endpoints used by the frontend.

- Dockerfile: Builds the client backend container.
- README_API.md: Documentation for the client API endpoints.
- requirements.txt: Python dependencies (Flask, flask-cors, pycryptodome, charm-crypto).
- **frontend/**
 - src/App.jsx: The main React component. Contains all UI state management, polling logic, and render logic for the login screen, sidebar (users and groups), chat window, modals, invitation pop-ups, and system notifications.
 - src/api.js: Thin wrapper around `fetch()` that abstracts all HTTP calls to the client backend REST API.
 - index.html: HTML entry point.
 - vite.config.js: Vite build configuration.
 - package.json: Node.js dependencies (react, motion/react, lucide-react, tailwindcss).
- Dockerfile: Main Dockerfile for the client container.
- README.md: General documentation for the ClientContainer.

4.2 Server Implementation (server_api.py)

4.2.1 Initialization

The `RealTimeSocketServer` class is initialized with a host and port (default `0.0.0.0:5000`). During `__init__`, the Boneh-Franklin IBE system is set up:

- A `PairingGroup('SS512')` object is created – this defines the elliptic curve pairing group.
- `ibe.setup()` returns the Master Public Key (mpk) and Master Secret Key (msk). These persist for the lifetime of the server process.
- Two thread-safe dictionaries are maintained: `self.clients` mapping username to socket, and `self.groups` mapping group name to a set of member usernames.
- A single `threading.Lock()` protects both dictionaries.

4.2.2 Message Dispatch in `handle_client`

Each client connection runs in its own thread executing `handle_client`. The thread reads raw bytes from the socket into a string buffer and uses `json.JSONDecoder().raw_decode`

to parse complete JSON objects from the stream, handling partial reads and multiple concatenated JSON objects correctly.

Dispatching is based on the “type” field of the parsed JSON:

- “register”: Checks uniqueness of the requested username. On success, stores the socket in `self.clients`, calls `ibe.extract(msk, username)` to derive the user’s secret key, serializes MPK and SK using `objectToBytes`, Base64-encodes them, and sends a success response with the encoded keys and the current user list. Triggers a `broadcast_user_list()` call.
- “message”: Calls `forward_message(sender, receiver, payload)` which looks up the receiver’s socket and forwards the encrypted payload. If the receiver is offline, a system error message is returned to the sender.
- “create_group”: Validates that the group name is unique. Creates the group entry in `self.groups` with the creator as the initial member. Derives a group secret key with `ibe.extract(msk, group_name)`, serializes and Base64-encodes it, and sends a `group_created` response to the creator.
- “invite_group”: Verifies that the requester is a member of the group and that the target user is online. Delivers a `group_invitation` push message containing the group name and inviter username to the target’s socket.
- “join_group”: Adds the requesting user to `self.groups[group_name][‘members’]`. Derives the group secret key, encodes it, and sends a `group_joined` response to the new member.
- “group_message”: Verifies that the sender is a member of the named group. Injects the sender’s username into the payload, then iterates over all group members (excluding the sender) who are currently in `self.clients`, and forwards the encrypted payload to each.
- “request_users”: Calls `send_user_list(requester)` to deliver the current online user list to the requester only.

On any exception or client disconnect, the finally block removes the user from `self.clients` and triggers `broadcast_user_list()`.

4.3 Client Backend Implementation (client_api.py)

4.3.1 Global State

The client backend maintains the following module-level state, all protected by `config_lock`:

- `mpk, sk`: The IBE Master Public Key and the user's personal secret key.
- `client_socket`: The persistent TCP socket connected to the server.
- `local_messages_db`: An in-memory list of decrypted message objects awaiting retrieval by the frontend.
- `active_users`: The current list of online users, updated on every `user_list` push.
- `local_group_sks`: A dictionary mapping group names to their IBE secret keys.
- `joined_groups`: A list of groups the user has joined.
- `pending_invitations`: A list of pending group invitation objects.

4.3.2 Background Listener Thread

The `listen_for_messages()` function runs in a daemon thread started after a successful connection. It reads from the socket continuously using the same streaming JSON parser as the server. Based on the "type" field, it dispatches as follows:

- `user_list`: Updates `active_users`.
- `system`: Appends a system notification object to `local_messages_db` with `is_system: True`.
- `group_created`: Deserializes the group secret key with `bytesToObj`, stores it in `local_group_sks[group_name]`, and adds the group to `joined_groups`.
- `group_invitation`: Appends the invitation to `pending_invitations`.
- `group_joined`: Deserializes and stores the group secret key, adds the group to `joined_groups`, and removes the corresponding entry from `pending_invitations`.
- `group_message`: Reconstructs the IBE ciphertext dictionary from the JSON representation (handling bytes, integer, and element types via `Base64/bytesToObj`). Calls `ibe.decrypt(mpk, group_sk, ibe_ctxt)` to recover the AES session key. Decrypts the message using `AES.new(session_key, AES.MODE_EAX, nonce=nonce).decrypt_and_verify(tag)`. Appends the plaintext with `is_group: True` and `receiver: group_name` to `local_messages_db`.
- `message`: Same decryption procedure as group messages, but uses the user's personal `sk`. Appends the plaintext with `is_group: False`.

4.3.3 REST API Endpoints

Private Messaging Endpoints

- **[POST] /connect:** Connects to the server, sends the registration payload, receives and deserializes the MPK and SK, starts the listener thread, and returns the initial user list.
- **[GET] /status:** Returns the current connection state (connected, username, server_ip, server_port). Used by the frontend on startup to bypass the login screen if already connected.
- **[POST] /send:** Encrypts a private message and sends it to the server.
 1. Generates a 32-byte random AES session key.
 2. Encrypts the plaintext with `AES.new(key, AES.MODE_EAX).encrypt_and_digest(message)`.
 3. Encrypts the session key with `ibe.encrypt(mpk, target_username, session_key)`.
 4. Serializes the IBE ciphertext to a JSON-safe dictionary (converting pairing group elements to Base64).
 5. Sends the combined payload as a ‘message’ type TCP packet.
- **[GET] /messages:** Returns all buffered decrypted messages and clears the buffer (one-shot poll).
- **[GET] /users:** Returns active_users.

Group Chat Endpoints

- **[POST] /groups/create:** Sends a create_group socket message to the server with the requested group_name. The server response is handled asynchronously by the listener thread.
- **[POST] /groups/invite:** Sends an invite_group socket message to the server with the group_name and target_user. The server forwards the invitation to the target.
- **[POST] /groups/accept:** Sends a join_group socket message to the server. The server responds with the group secret key, which the listener thread stores.
- **[GET] /groups/info:** Returns joined_groups and pending_invitations.
- **[POST] /groups/send:** Encrypts a group message and sends it to the server.
 1. Checks that the user has the group’s secret key in local_group_sks.
 2. Generates a 32-byte random AES session key.
 3. Encrypts the plaintext with AES-EAX.

4. Encrypts the session key with `ibe.encrypt(mpk, group_name, session_key)` – using the group name as the IBE identity.
5. Sends a `group_message` type TCP packet with the group name and encrypted payload.

4.4 Frontend Implementation (App.jsx)

4.4.1 State Management

All application state is managed with React `useState` hooks:

- `currentUser, connectionInfo`: Identity and server connection details after login.
- `onlineUsers, joinedGroups`: Sidebar lists, refreshed every polling cycle.
- `messages`: Accumulated message history for the current session.
- `selectedUser, isGroupChat`: Which contact or group is currently open.
- `pendingInvitations, ignoredInvites`: Group invitation pop-up management.
- `systemNotifications`: Transient error toasts (auto-dismissed after 5 seconds).
- `isCreatingGroup, newGroupName`: State for the Create Group modal.
- `isInvitingToGroup, inviteUsername`: State for the Invite User modal.

4.4.2 Polling Loop

A `useEffect` hook starts a 2-second polling interval after a successful login. Each poll cycle:

1. Calls `GET /users` and updates `onlineUsers` (excluding the current user).
2. Calls `GET /messages`. System messages are separated from normal messages. Normal messages are appended to `messages` with a formatted timestamp. System messages trigger toast notifications.
3. Calls `GET /groups/info` and updates `joinedGroups` and `pendingInvitations`.

4.4.3 Group Chat UI Components

- **Groups sidebar section**: Renders a button for each group in `joinedGroups`. Clicking sets `selectedUser` to the group name and `isGroupChat` to true. A “+” icon opens the Create Group modal.

- **Create Group modal:** An animated overlay with a text input for the group name. On submit, calls `POST /groups/create` and immediately navigates to the new group's chat window.
- **Invite User modal:** Accessible from the active group chat header via the `UserPlus` icon. On submit, calls `POST /groups/invite` with the entered username.
- **Group invitation pop-up:** Rendered in the bottom-right corner using `AnimatePresence`. Displays the inviter name and group name. "Accept" calls `POST /groups/accept` and navigates to the group chat. "Decline" adds the group name to `ignoredInvites` to hide the pop-up without joining.
- **Message filtering:** The current chat window displays messages filtered by `is_group` flag and matching `receiver === selectedUser` for groups, or matching both sender and receiver for private chats.
- **Chat header:** Shows the group icon (`Users` component) for group chats, or the user's initial avatar for private chats. Displays the IBE identity used for encryption as a subtitle.

Chapter 5

Challenges

During the development of the socket-based chat application, several challenges were encountered that affected system reliability, security, and the correctness of cryptographic operations. These challenges can be grouped into four main categories.

5.1 Socket Connection and Message Framing

- **TCP stream framing:** TCP is a stream-oriented protocol – it does not preserve message boundaries. A single `recv()` call may return a partial JSON object, a complete object, or multiple concatenated JSON objects depending on network conditions and OS buffering. Early implementations that called `json.loads(data)` directly would fail on partial reads.
- **Solution:** Both the server and client backend implement a string accumulation buffer and use `json.JSONDecoder().raw_decode(buffer)` in a loop to consume exactly one valid JSON object per iteration and retain the remainder in the buffer for the next read.
- **Unexpected disconnections:** The server holds no persistent storage. If a client disconnects ungracefully (e.g., Docker container crash), the server may attempt to write to a closed socket, raising an exception. The `finally` block in `handle_client` ensures the user is removed from the active registry and a user list broadcast is triggered.

5.2 Multi-Threading Challenges

- **Race conditions on shared state:** Both the server and the client backend maintain shared in-memory dictionaries accessed by multiple threads concurrently. Without synchronization, simultaneous reads and writes could produce corrupted state or silent data loss.

- **Solution:** A single `threading.Lock()` (`self.lock` on the server, `config_lock` on the client backend) is acquired using a `with` statement around every access to shared data structures.
- **Hardware limitations:** Creating one OS thread per client limits scalability on constrained hardware. For the scope of this project (a small number of concurrent users) the per-thread model is sufficient, but it would become a bottleneck at scale.
- **Flask and background thread coexistence:** The Flask development server must be started with `threaded=True` to allow concurrent HTTP requests from the frontend while the background socket listener thread continues to run without blocking.

5.3 Identity-Based Encryption (IBE) Serialization

- **Non-serializable pairing group elements:** The IBE ciphertext produced by `charm-crypto` consists of Python objects that are instances of pairing group element classes – they cannot be directly serialized to JSON.
- **Solution:** Each element is inspected at runtime using `type()` string matching. Byte strings are Base64-encoded; `charm.core.math.integer` objects are converted to Python `int` and stored as strings; pairing group elements are serialized with `objectToBytes` and then Base64-encoded. The receiver reconstructs the ciphertext by reversing this process using `bytesToObject` and `integer(int(v))`.
- **Centralized PKG as a single point of failure:** The IBE scheme requires all secret keys to be derived by a single PKG (the server). If the server is compromised, the attacker obtains the MSK and can derive any user's or group's secret key. This is an inherent limitation of centralized IBE.

5.4 Group Chat Implementation Challenges

- **Group secret key distribution:** Unlike private messaging (where each user has a unique SK derived at registration), group members all need the same group SK. The challenge was ensuring that the key is delivered securely to each member at the correct point in the join flow, without storing it on the server after delivery.
- **Solution:** The server re-derives the group SK on demand using `ibe.extract(msk, group_name)` each time a new member joins. Since the IBE Extract function is deterministic given the same MSK and identity, the derived key is always identical for the same group name.
- **Asynchronous group key storage on the client:** When the frontend calls `POST /groups/create`, the server's response (`group_created`) arrives asynchronously via the TCP push

channel, not as an HTTP response. The listener thread processes it and stores the group SK before the user attempts to send a group message. A timing issue could arise if the user sends a message before the listener thread processes the key; the `/groups/send` endpoint guards against this by checking `local_group_sks` before encrypting.

- **Message filtering in the frontend:** The message buffer contains messages from both private chats and multiple groups. The frontend must correctly filter messages to display only those belonging to the currently selected chat. This is handled by checking both the `is_group` flag and the `receiver` field (which equals the group name for group messages).
- **Invitation pop-up management:** The same invitation may arrive in multiple polling cycles if the server pushes it repeatedly. The `ignoredInvites` list in the frontend prevents already-dismissed or accepted invitations from re-appearing.

Chapter 6

Future Improvements

The current implementation provides secure one-to-one private messaging and IBE-based group chat with invitation management. Several enhancements can be considered to improve scalability, usability, and security.

6.1 Attribute-Based Encryption (ABE) for Fine-Grained Access Control

- **Concept:** Replace the group IBE scheme with Attribute-Based Encryption (ABE), where decryption rights are governed by a policy over user attributes (e.g., role, department, clearance level) rather than a shared group identity.
- **Purpose:** ABE enables more expressive access control. For example, a message could be encrypted with the policy “member of CS440 AND joined after 2025”, and only users whose attributes satisfy this policy can decrypt.
- **Impact:** Eliminates the need to distribute a shared group secret key. Each user decrypts using their own attribute-bound private key, removing the risk of the shared key being leaked by any one member.

6.2 Persistent Message Storage

- **Concept:** Introduce a local encrypted database (e.g., SQLite with AES-encrypted fields) on the client backend to persist message history across sessions.
- **Purpose:** Currently, the in-memory message buffer is cleared on GET /messages and lost when the container restarts. Users cannot scroll back through past conversations.
- **Impact:** Improves usability significantly without compromising end-to-end encryption, since messages are stored only in decrypted form on the client machine.

6.3 Multi-Server Architecture and Load Balancing

- **Concept:** Deploy multiple server instances behind a load balancer, with a shared session store (e.g., Redis) to synchronize the user registry and group membership across instances.
- **Purpose:** The current single-server model creates a performance bottleneck and availability risk. A multi-server architecture distributes connection handling across nodes.
- **Impact:** Improves scalability and fault tolerance. If one server instance fails, clients can reconnect to another without the entire system going offline.

6.4 Decentralized Key Management with Multi-Authority ABE

- **Concept:** Replace the single PKG with multiple independent attribute authorities, each managing a subset of user attributes. No single authority possesses the complete key material.
- **Purpose:** The centralized PKG is the most significant security weakness of the current design. Multi-authority ABE distributes trust so that compromising a single server does not expose all user keys.
- **Impact:** Substantially reduces the attack surface and aligns the system with a zero-trust security model.

6.5 WebSocket-Based Real-Time Frontend

- **Concept:** Replace the 2-second HTTP polling loop in the frontend with a WebSocket connection to the client backend.
- **Purpose:** Polling introduces latency of up to 2 seconds between message receipt and display. A WebSocket push mechanism would deliver new messages to the UI immediately when they arrive in the listener thread.
- **Impact:** Improves perceived responsiveness and reduces unnecessary HTTP traffic, especially when no new messages are pending.

6.6 Group Management Features

- **Leave group:** Allow users to voluntarily leave a group. The server should update the membership list and, if needed, rotate the group secret key so that the departed member cannot decrypt future messages.

- **Group administrator roles:** Designate the group creator as an administrator with exclusive rights to invite or remove members.
- **Group discovery:** Allow users to browse and request to join public groups, rather than relying solely on direct invitations.

Chapter 7

Conclusion

This project set out to design and implement a secure, multi-user chat application using socket programming and Identity-Based Encryption (IBE). The primary objectives were to enable real-time communication between users while ensuring end-to-end confidentiality and authentication, and to extend the system with a fully functional group chat feature protected by the same IBE scheme.

Through the integration of a multi-threaded TCP socket server acting as a Private Key Generator (PKG), a Flask-based client backend performing all cryptographic operations, and a React browser frontend, the system successfully demonstrated both secure one-to-one private messaging and IBE-based group chat with invitation management.

The implementation achieved several key results. The server manages multiple concurrent client connections using one thread per connection, protected by a shared lock to prevent race conditions. Private messages are encrypted with a hybrid IBE + AES-EAX scheme: a random AES session key encrypts the message content, and the IBE scheme (Boneh-Franklin over the SS512 pairing group) encrypts the session key using the recipient's identity string as the public key. For group chat, the group name is used as the IBE identity: the server derives a dedicated group secret key on demand and distributes it to the group creator and to each user who accepts a group invitation. Any group member can then encrypt a message for the group identity, and all members who hold the group secret key can decrypt it independently. The server routes the encrypted payload to all online group members without ever seeing the plaintext.

During development, notable challenges were encountered. TCP stream framing required a streaming JSON parser with a string accumulation buffer to correctly handle partial reads and concatenated messages. Serializing IBE pairing group elements to JSON necessitated a runtime type-inspection approach with Base64 encoding and custom deserialization on the receiver side. The asynchronous nature of the server's push responses for group operations (create, join) required careful coordination between the Flask HTTP handler and the background socket listener thread. The centralized PKG model, while simplifying key management, remains the primary security limitation of the IBE approach.

Despite these challenges, the project delivered a functional and modular secure chat application that covers the core requirements of the CS440 Socket Programming Project: TCP socket programming, multi-threading, message broadcasting, private messaging, group chat, and end-to-end encryption. The experience reinforced practical skills in concurrent programming, network application design, and applied cryptography.

Looking ahead, the most impactful improvements would be replacing the polling-based frontend with a WebSocket push mechanism for true real-time updates, introducing persistent encrypted local storage for message history, and migrating from centralized IBE to a multi-authority Attribute-Based Encryption scheme to eliminate the single point of failure in the PKG. Group management features such as voluntary leave with key rotation and administrator roles would also enhance the practical usability of the group chat system.

In summary, the project met its objectives and produced a working demonstration of how socket programming and identity-based cryptography can be combined to build a secure, real-time, multi-user chat application with both private and group communication capabilities.